

Graph & Scenario Construction

How the asset graph is built — physical nodes, abstract aggregates, and the partition tree

Master's Thesis — *Asset-Centric Temporal Graphs for Crude-Oil Logistics: Schema Design and Consistency Guarantees*

Pedro Porfirio · NOVA IMS · Universidade Nova de Lisboa

Supervisor: Professor Flavio Pinheiro

Document: **Graph Construction** · v1.0 · 2026-05-14

1. The two-layer model

The framework keeps two layers separated cleanly. The **persistent asset graph** is the fixed, superset physical topology — every node and every feasible edge defined once and only once. It does not change when data sources change. The **scenario state** is derived at load time from the persistent asset graph plus the per-scenario rows in `variable_assignments`; this is what downstream consumers query.

Why two layers. The empirical data the framework consumes (EIA monthly series, for the U.S. crude case) covers different subsystems at different resolutions and changes vintage as new series are published. A representation that adapts to data shape by rewriting nodes and edges would force every consumer to keep up with that vintage. The two-layer model puts the volatility in `variable_assignments` instead: the asset graph stays stable; the scenario state regenerates when the assignment rows change. (Earlier framings of this idea introduced a “coverage contract” object as a separate primary structure. In the implementation, the contract is mechanically just the assignment table tagged with a `scenario_id` — no separate object exists.)

Concretely, if per-refinery operating stocks become available, the asset graph does not change. The affected variables are re-pointed from PADD-level TS to refinery-level TS in `variable_assignments`; the listed refineries are inferred as *authoritative* by `v_node_status` (rather than the *collapsed* they were previously). The refineries already exist as nodes with inventory variables.

2. The starter asset graph

The starter asset graph (target: U.S. crude oil, January 2015 onwards) carries **240 assets**: 197 physical and 43 abstract aggregates (including 4 boundary nodes). They connect via 409 directed flow edges, and they carry 1,830 variables under the active scenario `starter_us_crude_2015_2025`.

Every node has a *kind* (physical or abstract), a *node_class* (production, infrastructure, observational), a *node_subtype* (refinery, gathering, pipeline, region_view, ...), and a *starter_status* (authoritative, collapsed, derived). Geographic metadata (lat/lon, state, county, PADD) is stored separately in `oil_network.locations` to keep labelling distinct from structure (Principle 2.7).

3. Physical nodes

Physical nodes represent real, named assets on the ground. They carry geographic coordinates, capacity, configuration, and flow edges. Adding a new physical node usually means a real operational change: a new refinery, a new pipeline, a new export terminal coming online. The starter graph has the following physical inventory:

Subtype	Count	Examples / role
state_sub_basin	5	bakken_nd, bakken_mt, permian_tx, permian_nm, eagle_ford_tx
state_conventional	7	oklahoma_conventional, california_conventional, wyoming, ...
state_residual	5	padd1_other ... padd5_other (covers the un-named tail)
offshore_region	1	gulf_of_america
gathering	6	ans, bakken_nd, bakken_mt, eagle_ford, permian_nm, permian_tx
origin_terminal	6	midland, wink, crane, johnsons_corner, three_rivers, valdez
storage_terminal	10	Cushing (hub + 3 sub-operators), Patoka, Houston, ...
spr_site	4	bayou_choctaw, big_hill, bryan_mound, west_hackberry
refinery	115	19 named + 5 PADD residuals + sub-aggregates (24 active)
import_terminal	8	clearbrook, 4 PADD imports_agg, 2 Canadian imports_agg, padd2_xstate...
export_terminal	12	Ingleside, Houston-agg + 3 subs, Nederland, LOOP, 5 exports_agg
pipeline	37	20 US-internal + 4 cross-border + 12 inter-PADD aggs + the new bakken xstate
foreign_export_destination	1	boundary sink: where exports physically leave
foreign_production_aggregate	2	canadian_oil_sands, foreign_supply

A few of these node categories warrant comment.

State residuals (padd1_other ... padd5_other) are physical in the sense that they aggregate real but un-named refineries within the PADD. They exist because EIA publishes a PADD total that exceeds the sum of the named facilities, and the difference must be attributed somewhere. They will be replaced by named facilities as data granularity improves — each residual carries an explicit list of the refineries it stands in for, so the replacement is traceable.

Inter-PADD and import/export aggregates (added in the tenth pass) are physical nodes in the technical sense — they have `kind = 'physical'`, an asset row, and concrete flow edges — but they aggregate over groups of real pipelines and terminals rather than naming a single facility. They exist to give the partition tree a same-type child to point at, and they are the natural home for future per-grade or per-operator decomposition.

The Bakken cross-state connector (`pipe_bakken_xstate`, added in the eleventh pass) is a bidirectional pipeline-type node connecting `bakken_nd_gathering` and `bakken_mt_gathering`. It models the real midstream pooling between the two state halves of the Bakken (operated by Hess, Crestwood, Energy Transfer). All four of its flow variables are latent in the starter scenario, because no public TS measures inter-state Bakken gathering flow; the constraint $\Sigma I = \Sigma O$ at the connector follows from the pipeline node-type defaults.

4. Abstract nodes (observational aggregates)

Abstract nodes do not represent physical assets. They are aggregation views over physical nodes — the substrate for partition rollups, data attribution at coarser scales than the physical layer, and Chapter-5 consistency claims. They have `kind = 'abstract'` and no flow edges of their own; their relationships to physical nodes are through `formula_inputs`.

Subtype	Count	Role
region_view	6	usa_view, padd1_view ... padd5_view. The geographic partition spine.
refining_district_view	10	EIA refining-district rollups (R3A, R3B, ...).
state_view	2	texas_state_view, montana_state_view — rollups by state used for sub-PADD cross-checks.
observational_aggregate	4	spr_total, basin aggregates (permian, bakken, eagle_ford) when used as constraint rollups.
usa_subtotal_view	1	usa_lower48_excl_gom_view — STEO subtotal.

Why region_view nodes are abstract. A PADD is an administrative label, not a physical asset. There is no piece of equipment one can point at and call “PADD 2”. EIA reports aggregated data at this scale, and the framework needs a node that can carry that aggregated data, but it is structurally different from a refinery or a pipeline. Treating it as physical would invite confusion at the edges (does PADD 2 have an outflow to PADD 3, or is that outflow the sum of the underlying pipelines?). The framework picks the latter: PADD-view outflow is alias-derived from the corresponding inter-PADD aggregate’s outflow, never TS-bound directly.

Why refining_district_view nodes are abstract. EIA refining districts (PADD 3A, 3B, 3C, ...) group refineries for reporting; they are not co-owned by a single operator, do not share custody transfer mechanics, and do not have a single physical address. They exist to host the district-level consumption series that EIA publishes and to enable the sum-over-children identity that PADD-level consumption must equal the sum of district consumptions.

5. Boundary nodes

Four nodes sit on the boundary of the modelled system. They are physical in the sense that they represent real entities or aggregates, but they are not full participants in the partition.

Node	Subtype	Role
foreign_supply	foreign_production_aggregate	All non-Canadian foreign crude flowing into U.S. import terminals. Boundary source; foreign-side mass balance is not modelled.
canadian_oil_sands	foreign_production_aggregate	All Canadian crude entering U.S. via Mainline, Keystone, Express+Platte, TMX. Boundary source.
foreign_export_destination	foreign_export_destination	Boundary sink. All U.S. crude exports flow into this node; we do not model where they go from here.
spr_total	observational_aggregate	Sum of the four SPR sites. Constraint node, used for cross-checks rather than as a partition member.

6. Variables: the authoritative data model

Variables are the unit of data attribution. Every node carries one variable per (variable_type, commodity, related_node) tuple. The seven variable types are:

Type	Symbol	Description	Relational?
production	P	Crude produced at this node.	no
consumption	C	Crude consumed at this node (refinery throughput).	no
inventory	S	Crude stocks held at this node.	no
balancing_item	B	Reporting residual; absorbs systematic discrepancies.	no
inflow	F_{in}	Crude flowing in from a specific related_node.	yes
outflow	F_{out}	Crude flowing out to a specific related_node.	yes
phi	ϕ	Reference variable (used for cross-node alias chains).	yes

Each variable carries either a *timeseries_id* (TS-bound, observed) or a *formula* (derived), never both — enforced by the schema-level CHECK constraint `num_nonnulls(timeseries_id, formula) = 1`. This is the schema-level expression of Principle 2.8.

The variables collection is the authoritative data model. Every other view of the graph — the edge list, the partition tree, the resolution hierarchy — is derived from it. There is no second source of truth to keep in sync.

7. The partition tree

The partition tree is the same-type aggregation hierarchy that drives the Chapter-5 consistency claim. A node M is a partition child of node N if some variable on N has `formula_inputs` referencing a same-type, same-commodity, same-related-node variable on M .

The *same-type* qualifier is critical. Aggregation links (`padd_view.P =  $\Sigma$  basin.P`) connect variables of the same type. Cross-edge aliases (`padd_view.inflow = alias(other_padd.outflow)`) connect variables of *different* types — these are inter-PADD references, not partition relations. Filtering on type cleanly distinguishes the two.

The starter scenario carries 265 partition edges across 28 distinct parents and 197 distinct children. The active partition spine runs:

```

usa_view
  ■■■ padd1_view
  ■   ■■■ refining districts (RAP, REC)
  ■   ■■■ padd1_imports_agg          ← MCRIPP12 - MCRIPP1CA2
  ■   ■■■ padd1_canadian_imports_agg ← MCRIPP1CA2
  ■   ■■■ padd1_exports_agg          ← MCREXP12
  ■   ■■■ inter_padd_1_to_2_agg, inter_padd_1_to_3_agg
  ■   ■■■ padd1_other (residual)
  ■■■ padd2_view
  ■   ■■■ refining districts (R2A, R2B, R2C)
  ■   ■■■ bakken_nd, oklahoma, padd2_other
  ■   ■■■ cushing_hub (+ 3 operator sub-terminals), patoka_hub
  ■   ■■■ bakken_nd_gathering (inventory-child, post-11th pass)
  ■   ■■■ padd2_canadian_imports_agg ← MCRIPP2CA2
  ■   ■■■ padd2_exports_agg          ← MCREXP22
  ■   ■■■ inter_padd_2_to_{1,3,4}_agg + receiver-side aliases
  ■■■ padd3_view, padd4_view, padd5_view (analogous)
  ■
  ■■■ boundary nodes (sit outside the partition):
      foreign_supply, canadian_oil_sands, foreign_export_destination, spr_total

```

Partition closure is the algebraic identity that the audit verifies: for every parent node N and every variable_type T , the own value of N 's T variable equals the sum of N 's partition children's T variables. This holds (within rounding) across all PADDs and at `usa_view` in the current state.

8. Scenarios and authoritative declarations

A scenario is a pair: the persistent asset graph plus a set of `variable_assignments` rows tagged with the `scenario_id`. Different scenarios share the asset graph and the TS data; they differ in their assignment rows.

The schema-level `CHECK num_nonnulls(timeseries_id, formula) = 1` means each assignment row either binds the variable to a TS (authoritative) or to a formula (derived) — never both, never neither. The set of assignments under a scenario therefore encodes, for every variable, exactly which level is authoritative and which is derived. There is no separate “coverage contract” structure; the per-variable choices are the contract.

The starter scenario’s authoritative declarations summarise as:

Subsystem	Authoritative level	Reason
Permian, Bakken	basin aggregate	EIA publishes only at basin level
Eagle Ford	state-basin	Single state, equivalent to basin
Gulf of America, Alaska	physical node	Reported separately by EIA
Rest of L48	residual aggregate	EIA publishes only the residual
SPR	site level	EIA PSM Table 38 publishes per site monthly
Cushing	hub level	Operator sub-terminals collapsed; storage report at hub
Houston export	aggregate	Facility sub-terminals collapsed
Stocks	PADD level (auxiliary at sub-)	MCRSFP+MCRRSP at PADD; sub-decomposition would double-count
B (balancing item)	PADD + USA	MCRUA series, ninth-pass promotion to TS-observed
Inter-PADD flows	PADD-aggregate level	MCRMP_*_*_* at the combined aggregate, per-pipe latent

Cross-scenario consistency (the Chapter-5 contribution) tests that the same TS data, evaluated under progressively finer assignment sets, produces consistent aggregate values. The current implementation has `starter_us_crude_2015_2025` as the only scenario; `starter_basin` (a coarser contract) is queued for the next pass.

10. End-to-end pipeline: load → resolve → render

Every run of the framework follows three phases. This section walks through each, naming the script and the tables it writes.

10.1 Load the database (one-time + ad-hoc migrations)

Phase 1 populates the persistent asset graph and per-scenario `variable_assignments`. The seed file `clean/config/asset_graph.json` is the starting point; migration scripts incrementally evolve the graph (adding aggregates, splitting nodes, fixing routes) by writing directly to the base tables.

Step	Script	Writes to
1.1	<code>load_asset_graph.ipynb</code>	<code>assets</code> , <code>nodes</code> , <code>locations</code> , <code>variables</code> — the persistent asset graph from JSON
1.2	<code>assign_eia.ipynb</code>	<code>timeseries</code> , <code>timeseries_data</code> — EIA series catalogue + monthly facts
1.3	<code>assign_formulas.ipynb</code>	<code>variable_assignments</code> — per-scenario TS or formula recipe per variable
1.4	<code>populate_node_type_defaults.py</code>	<code>node_type_default_formulas</code> — default recipes per node type
1.5	Migration scripts (<code>add_*.py</code> , <code>split_*.py</code> , <code>apply_*.py</code> , <code>repoint_*.py</code> , <code>*pass_*.py</code>)	<code>assets</code> / <code>nodes</code> / <code>variables</code> / <code>variable_assignments</code> — topology + assignment evolutions
1.6	<code>refresh_views.py</code> <code>--structural</code>	Refreshes L2 + L3 materialised views (<code>v_formula_input_links</code> , <code>v_aggregation_edges</code> , <code>v_flow_edges</code> , <code>v_partition_tree</code> , <code>v_node_status</code>) after any change to <code>variable_assignments</code>

The base tables under `oil_network` after phase 1: `assets`, `nodes`, `locations`, `variables`, `variable_assignments`, `node_type_default_formulas`, `scenarios`, `scenario_node_role`, `timeseries`, `timeseries_data`, `commodities`, `variable_types`, `node_types`, `graphs`. Plus the materialised views `v_effective_assignments` (L1, regular), L2 / L3 mat views listed above.

10.2 Use the solver (every assignment change)

Phase 2 evaluates every variable at every observation date, applying the dispatch rules described in *Resolver Walkthrough*.

Step	Script	Writes to
2.1	<code>resolve_scenario.py</code>	<code>scenario_resolver_runs</code> — one audit row per invocation (started_at, completed_at, duration_ms, n_assignments, n_rows_written, dispatch_stats as JSONB, free-text notes)
2.2	(same script)	<code>scenario_resolved_values</code> — one row per (scenario, variable, observation_date) with value / source / formula_used / timeseries_id / run_id
2.3	(automatic, end of <code>resolve_scenario</code>)	Calls <code>refresh_views.refresh_analytic()</code> → rebuilds L4 mat views (<code>v_node_balance_check</code> , <code>v_aggregation_consistency</code> , <code>v_inventory_changes</code> , <code>v_aggregate_balance</code> , <code>v_node_pcisob</code>)

Audit scripts that read post-resolve state: `audit_partition_gaps.py` (gap.I/gap.O closure check),
`audit_ts_binding_uniqueness.py` (1-TS-1-variable invariant),
`verify_balance_cells.py` (cell-by-cell P/C/I/O/B/S sums).

10.3 Show the data on HTMLs

Phase 3 generates the visualisation HTMLs. Every renderer goes through `NetworkGraph` (the in-memory engine) which loads from the L2 / L3 / L4 mat views; no renderer issues raw SQL against base tables. Each HTML carries a metadata beacon (`oilnet-artefact`) naming the resolver run that produced it, and an audit row is written per regeneration.

Step	Script	Output / writes to
3.1	<code>regenerate_htmls.py</code>	Orchestrator: scans <code>clean/outputs/html/</code> , compares each HTML's beacon against the latest <code>scenario_resolver_runs.run_id</code> , calls only stale renderers (or all with <code>--force</code>)
3.2	<code>make_balance_resolver_ui.py</code>	<code>oil_network_balance_resolver.html</code> — balance equation per node, drill-down by partition tree
3.3	<code>make_hierarchy_resolver_ui.py</code>	<code>oil_network_hierarchy_resolver.html</code> — full asset-graph tree with variable inspector
3.4	<code>make_partition_map.py</code>	<code>oil_network_partition_map.html</code> — click-to-drill geographic map of the partition tree
3.5	<code>make_node_neighbors_map.py</code>	<code>oil_network_node_neighbors.html</code> — single-node flow-edge explorer
3.6	<code>make_map_resolver_ui.py</code>	<code>oil_network_map_resolver.html</code> — flat geographic map of all physical assets + edges
3.7	Each renderer calls <code>render_util.s.write_html()</code>	<code>scenario_html_artefacts</code> — audit row per generation (<code>scenario_id</code> , <code>run_id</code> , <code>view_name</code> , <code>file_path</code> , <code>file_size_bytes</code> , <code>generated_at</code> , <code>notes</code>)

After step 3.1 a single SQL query answers *which run produced which HTML*: `SELECT DISTINCT ON (view_name) view_name, run_id, generated_at FROM scenario_html_artefacts ORDER BY view_name, generated_at DESC.`

11. Where the framework stands today

After twelve passes of refinement, the partition audit reports clean closure on both sides for every PADD view and for `usa_view`:

Node	own.I	bdy.I	gap.I	own.O	bdy.O	gap.O
<code>usa_view</code>	6,557	6,557	0	3,752	3,752	0
<code>padd1_view</code>	705	705	0	92.2	92.2	0
<code>padd2_view</code>	4,503	4,503	0	2,426	2,426	0
<code>padd3_view</code>	3,631	3,631	0	4,264	4,264	0
<code>padd4_view</code>	552.7	552.7	0	987.5	987.5	0
<code>padd5_view</code>	1,183	1,183	0	—	0	0

TS-binding uniqueness is clean (80 distinct TS in 80 bindings, 1:1 attribution per Principle 2.8). Zero unresolved variables across $1,830 \text{ assignments} \times 156 \text{ monthly observations} = 285,480$ (variable, date) pairs. The framework's partition + Principle-2.11 latent-allocation machinery is end-to-end consistent.

This concludes the framework's representational contribution. The remaining work for the thesis lies in the Chapter-5 validation experiments (cross-scenario consistency, the observed-vs-closure-B view, the named-event Chapter 5 Claim 4 chart) and the case-study narrative in Chapter 6.